

校园二手交易平台数据库大作业 – 项目说明文档

电计 2302 20231072030 袁善

一. 在线访问网址:

<https://campus-market-db-ys.vercel.app/>

(测试账号: u001, 密码: Campus123!)

二. 从代码到最终网址

本项目基于 Vercel 平台与 Vercel Postgres 数据库进行全云端部署, 具体构建与部署步骤如下:

1. 项目创建与仓库关联: 在 Vercel 控制台创建全新项目, 并授权导入本项目的 GitHub 代码仓库。
2. 云数据库实例配置: 在 Vercel 的 Storage 面板中创建 Serverless Postgres 数据库实例, 并获取数据库连接串 (Connection String)。
3. 环境变量配置: 在 Vercel 部署设置中, 添加以下环境变量以确保前后端正常通信:
 - 1) POSTGRES_URL = <获取到的数据库连接串>
 - 2) AUTH_SECRET = <自定义的随机长密钥, 用于鉴权>
4. 数据库初始化 (按序执行): 在 Vercel Postgres 的 Data 面板 (或使用本地连接工具) 中, 严格按照以下顺序执行 SQL 脚本, 以完成数据库结构的搭建与数据的注入:
 - 1) sql/01_schema.sql (创建表结构及外键约束)
 - 2) sql/02_seed.sql (插入作业要求的初始数据)
 - 3) sql/03_views.sql (创建相关视图)
 - 4) sql/04_business_logic.sql (创建业务逻辑相关的存储过程/函数)
 - 5) sql/05_security_roles.sql (配置数据库安全与角色权限)
5. 触发部署: 完成上述配置后, 推送代码至 main 分支或在 Vercel 点击 Deploy 触发项目构建。
6. 访问验证: 构建完成后, 打开 Vercel 提供的生成域名, 确认页面及数据库交互功能正常可用。

三. 网页截图及运行结果示例说明

1. 基础页面展示

- [图 1: 首页展示] 平台首页, 包含全局顶部导航栏、核心功能入口以及系统概况的统计卡片。
- [图 2: 商品列表页] 展示所有商品信息, 顶部配备筛选栏, 支持按状态、价格区间、类别、卖家等多维度进行数据检索。
- [图 3: 用户列表页 & 订单列表页] 分别展示数据库中 User 表的用户信息, 以及 Orders 表的订单记录, 订单页同样具备按商品名、卖家、日期等字段筛选的功能。
- [图 4: 查询展示页] 展示数据聚合与视图内容。

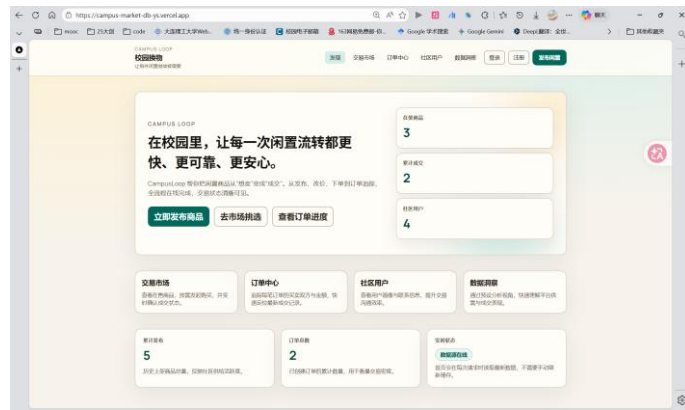


图 1

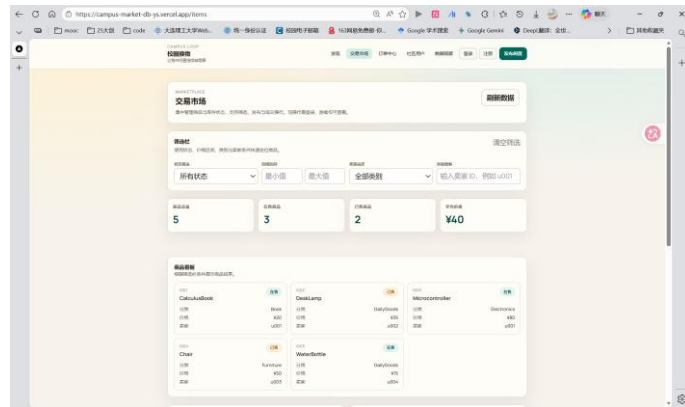


图 2

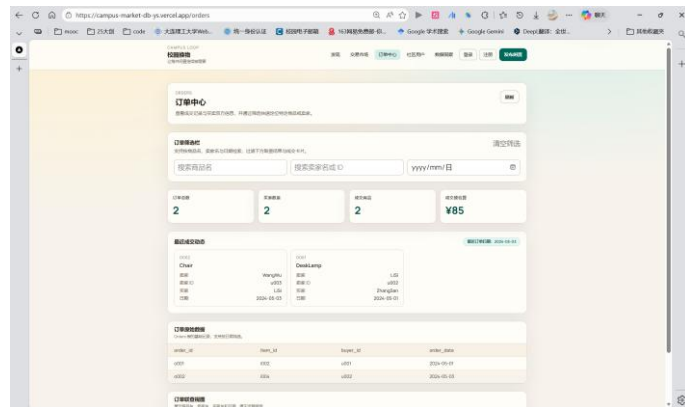
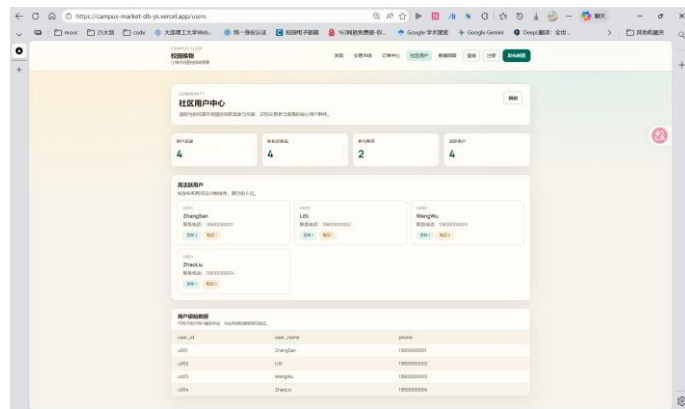


图 3

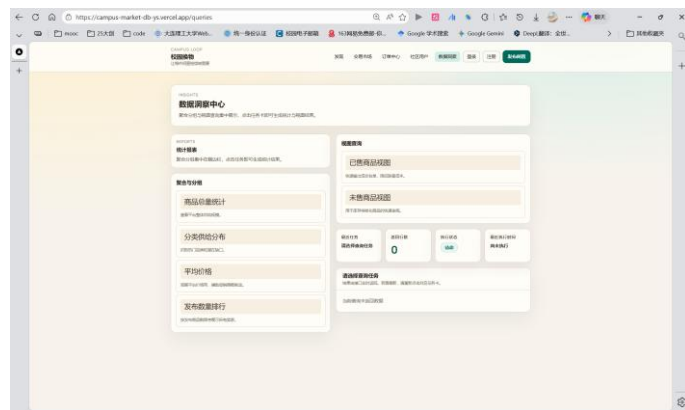


图 4

2. 数据操作与业务逻辑演示 (以 u001 身份登录测试)

- [图 5: 插入新商品] 演示发布新商品功能。用户填入商品信息 (i006, desk, 35) 后，成功写入数据库并实时展示在商品列表中。
- [图 6: 修改商品价格] 以卖家 u001 身份，将商品 i001 (CalculusBook) 的价格由 20 成功修改为 10，列表中价格实时更新。
- [图 7: 删除未售出商品] 演示 DELETE 操作。用户 u001 删除刚才发布的未售出商品 i006，操作后商品从列表中彻底消失，满足作业要求。
- [图 8: 购买商品 (业务逻辑实现)] 用户 u001 购买了 u004 发布的商品 i005。操作完成后，订单列表中新增了一条对应记录，且商品列表中该商品状态已变更为“已售出”。

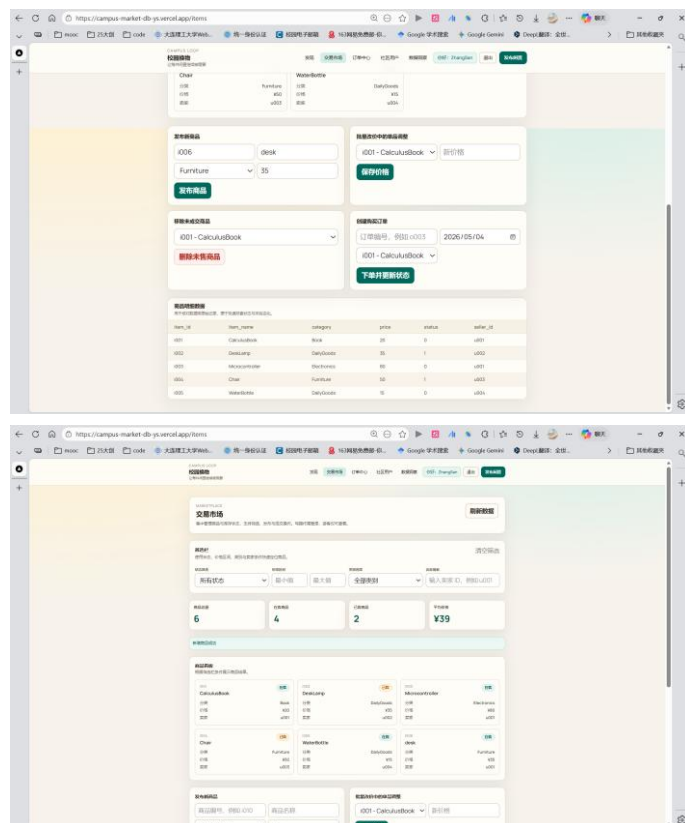


图 5

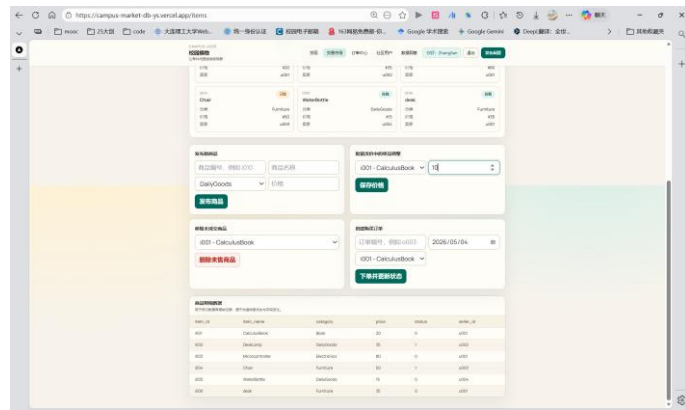


图 6

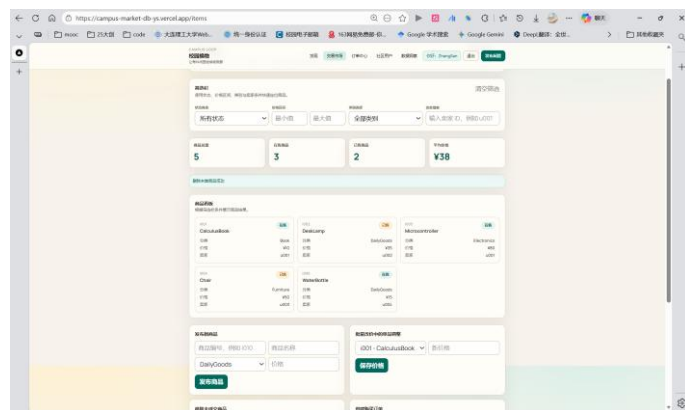
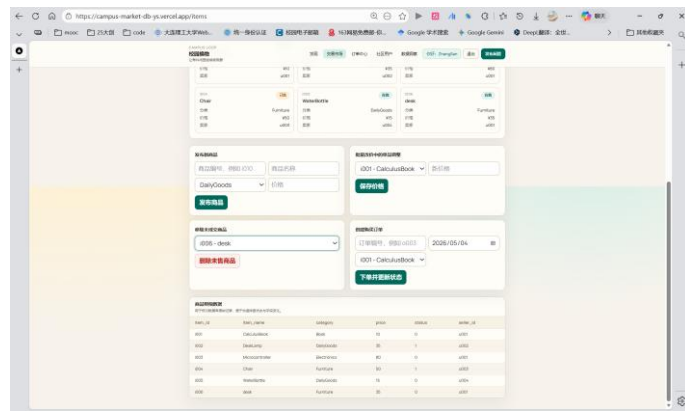
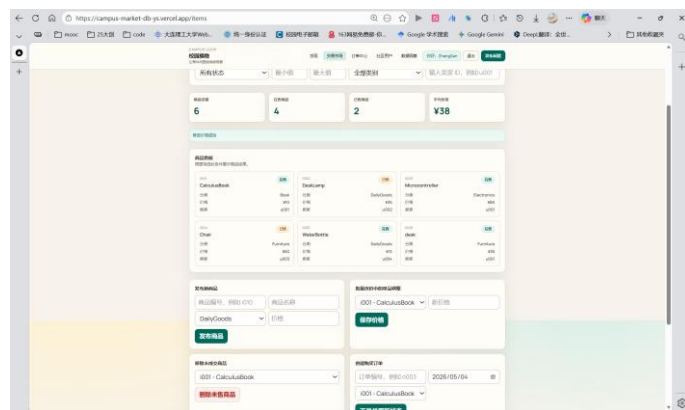


图 7

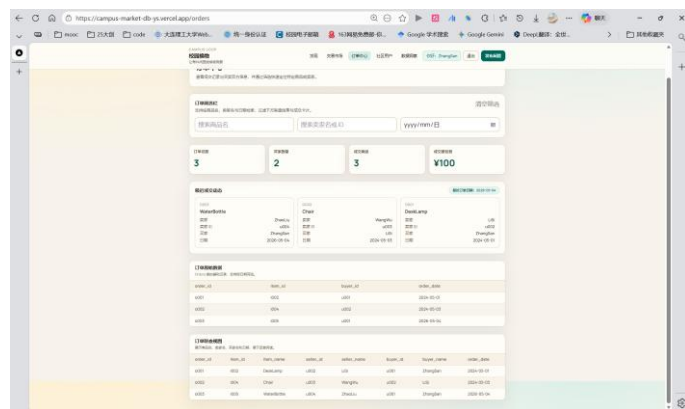
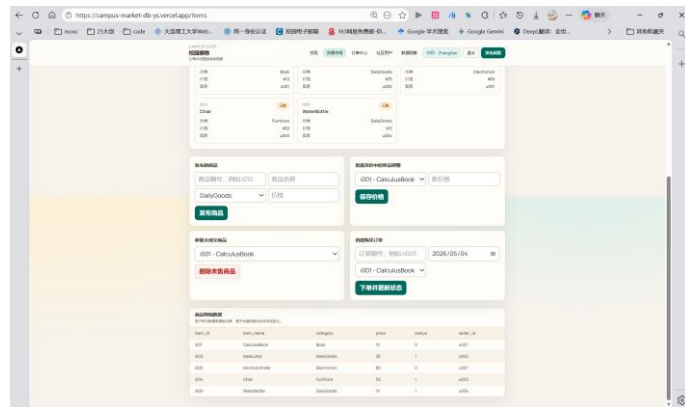
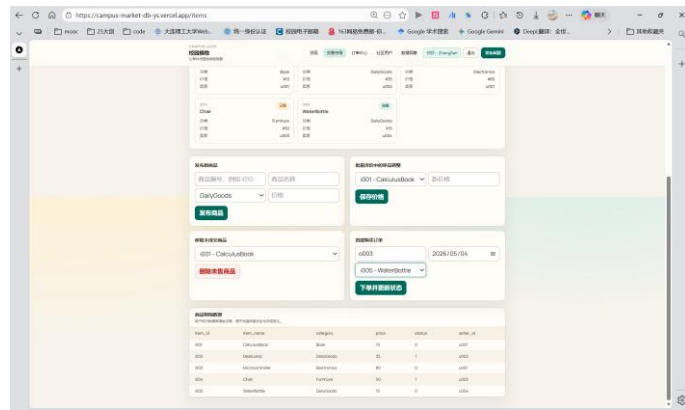


图 8

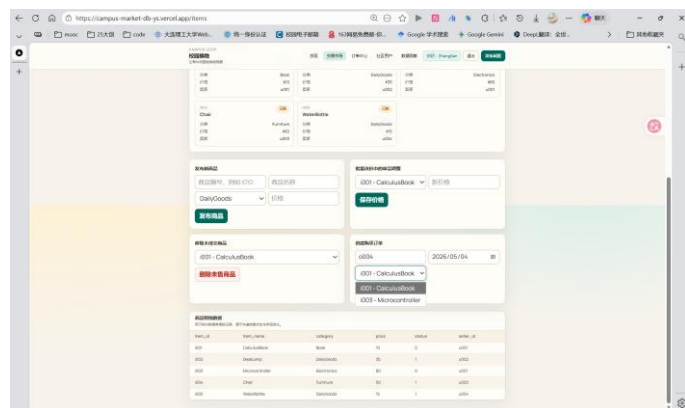


图 9

- [图 9: 购买已售商品失败拦截] 一致性约束演示。尝试购买状态为“已售出”的商品时，系统下拉栏限制/操作被拒绝，确保同一商品不会被重复交易。
- [图 10: 未登录安全拦截] 安全性测试。在未登录（游客身份）状态下尝试进行写操作（增删改），系统有效拦截并弹出报错提示，保证数据安全。

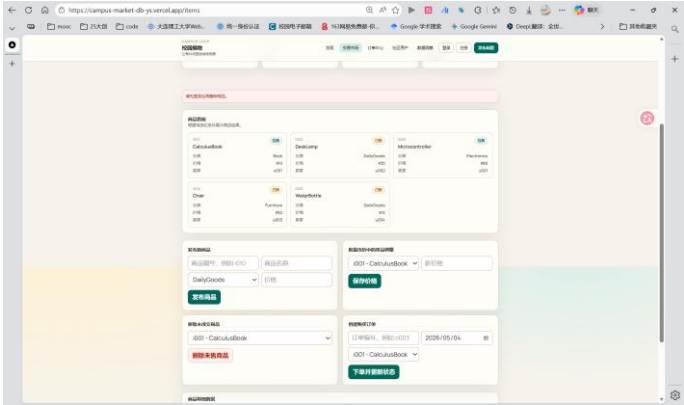


图 10

3. 数据查询（基础查询与连接查询）

- [图 11-14: 基础查询组合] 通过 Filter Bar 成功筛选出：1. 所有未售出商品；2. 价格 > 30 的商品；3. “生活用品 (DailyGoods)” 类商品；4. 由用户 u001 发布的所有商品。
- [图 15 - 17: 连接查询展示] 通过多表联查实现：已售商品与买家姓名的对应展示；订单明细视图（商品名+买家名+交易日期）；以及筛选卖家 u001 的商品被购买的状态。

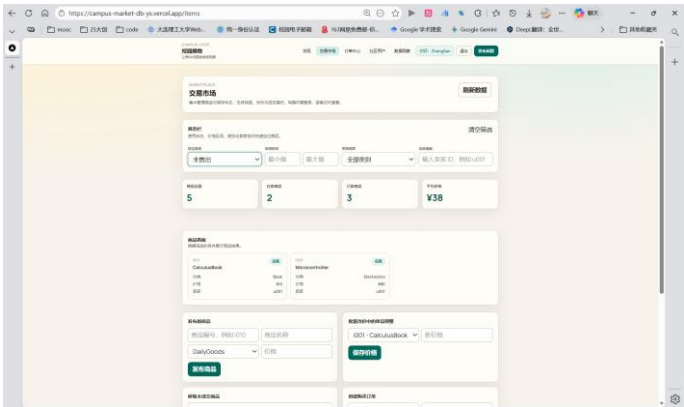


图 11

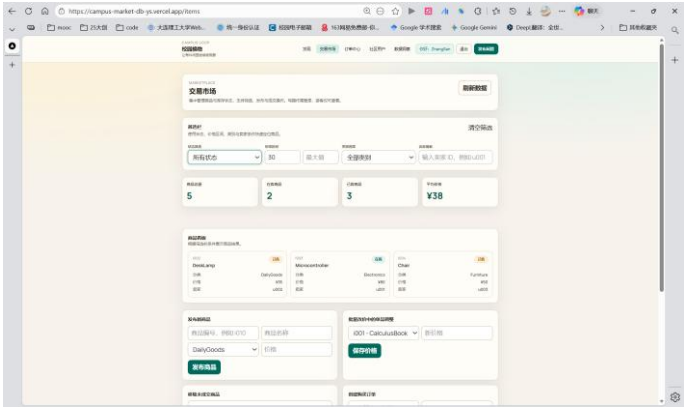


图 12

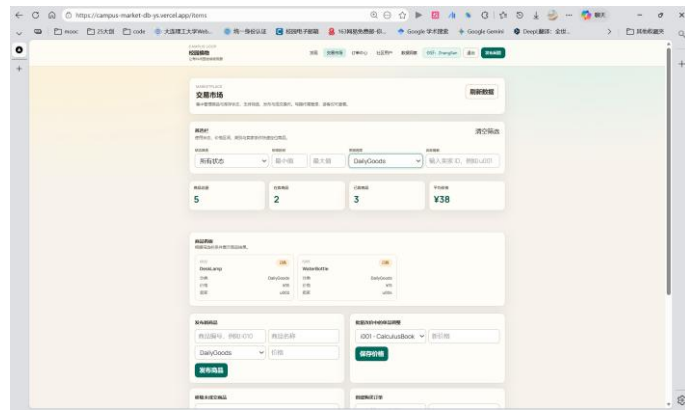


图 13

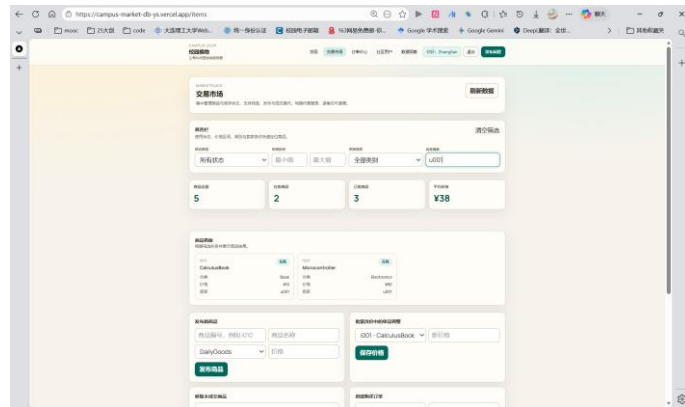


图 14

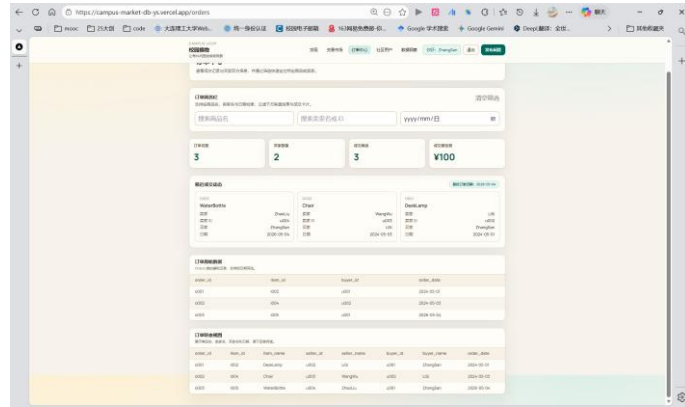


图 15

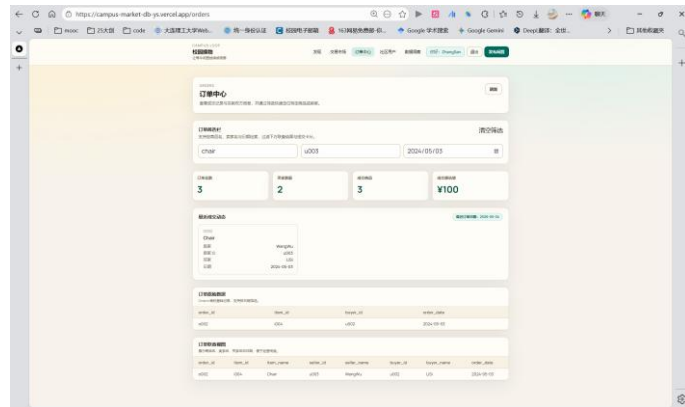


图 16

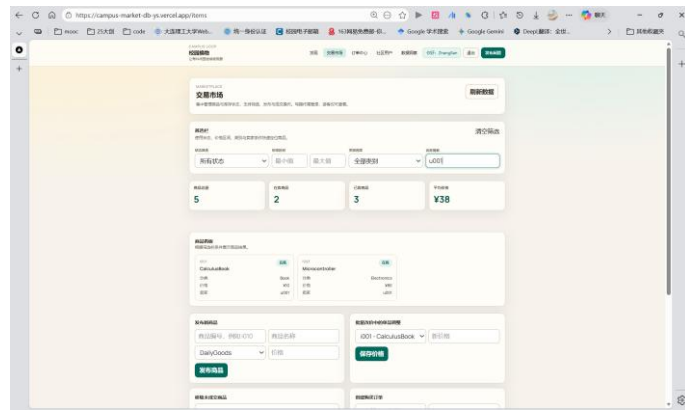


图 17

4. 聚合、分组与视图展示

- [图 18 - 21: 聚合与分组] 展示统计商品总数；统计每类商品数量；计算平均价格；发布商品数量排行（按发布数降序列出所有卖家）。
- [图 22 - 23: 视图展示] 通过预设的 SQL 视图（已售商品视图、未售商品视图）直接渲染数据。

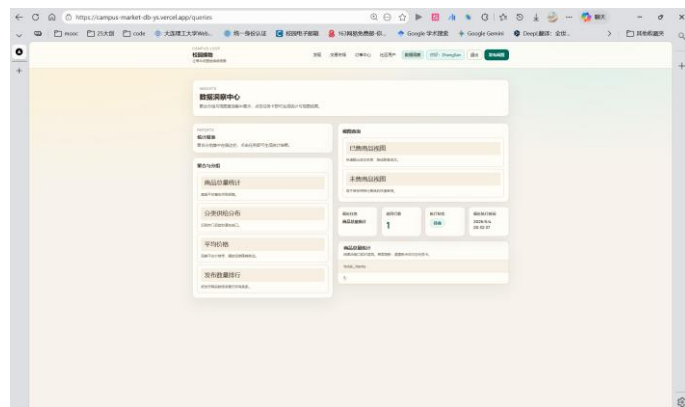


图 18

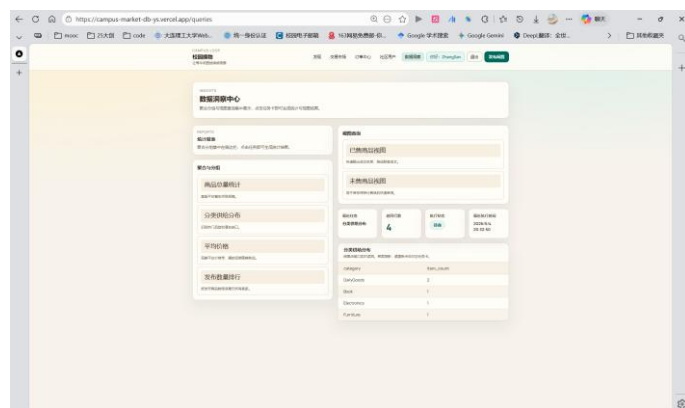


图 19

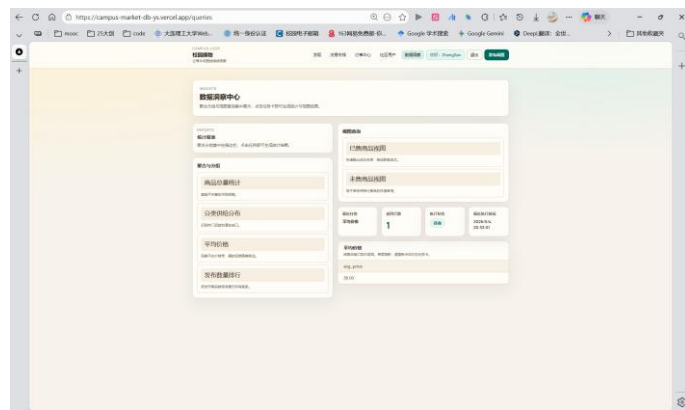


图 20

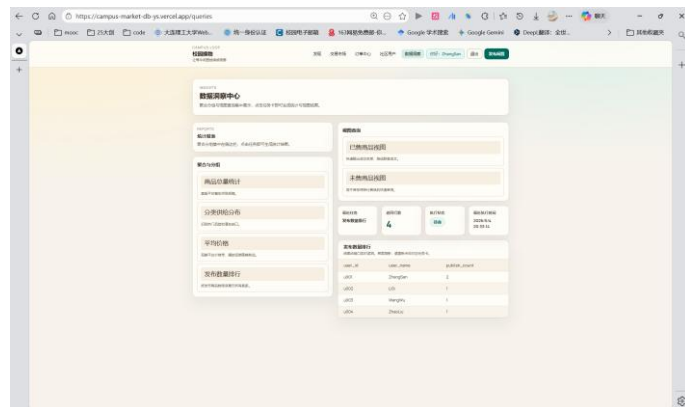


图 21

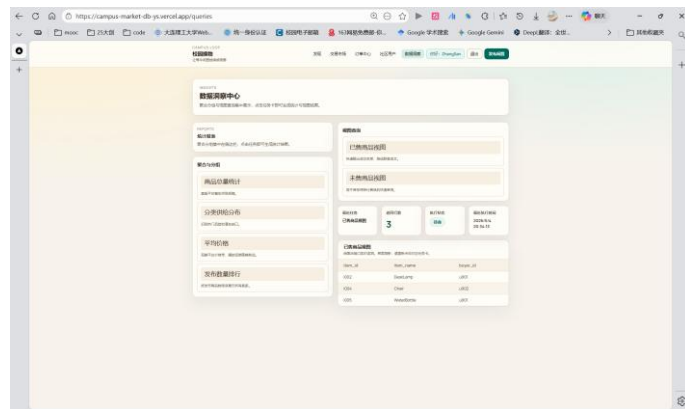


图 22

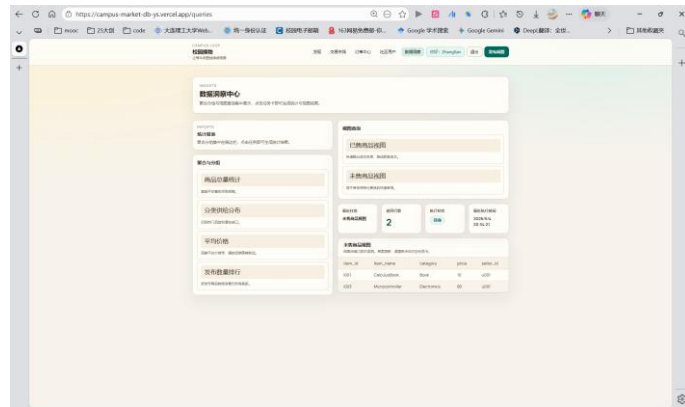


图 23

四. 简答:

(八) 安全性保障机制

1. 如何防止普通用户删除数据?

数据库层面 (RBAC 角色权限控制): 在数据库中建立多套用户角色 (Role)。区分 “只读角色” 和 “读写角色”。严格控制权限, 仅赋予特定管理员角色执行 DELETE 表的权限, 普通用户的数据库连接角色仅授予 SELECT, INSERT, UPDATE 权限。

应用层面: 所有涉及写操作 (增、删、改) 的 API 接口必须通过 Token (如 JWT) 鉴权, 强制要求用户登录。并在接口层校验操作对象是否属于当前用户 (如只能删除 seller_id 是自己的商品)。

2. 如何限制用户只能查询数据?

针对游客或未授权用户, 后端应用在处理其请求时, 仅开放 GET 请求类 (读接口) 的路由。若游客尝试触发 POST/PUT/DELETE 接口, 应用层将直接返回 401 Unauthorized 拒绝访问。

(九) 并发与恢复机制

1. 两个用户同时购买同一商品会出现什么问题?

若无并发控制, 会出现超卖现象。两个用户同时读取到商品 status=0 (未售出), 然后系统分别为两人在 orders 表中创建记录, 并将商品更新为已售出。最终导致一个商品被卖了两次, 违背了一致性规则。

2. 如何解决?

事务控制 + 行级悲观锁, 将 “检查状态+插入订单+更新状态” 这三个步骤包裹在一个数据库事务中。

在查询商品状态时使用 SELECT ... FOR UPDATE 语句。这会对该商品的数据行加锁, 第二个尝试购买的用户必须等待第一个用户的事务提交 (或回滚) 后才能读取该行数据, 此时状态已变为 1, 从而被业务逻辑拦截。

在 Orders 表中, 对 item_id 字段设置唯一性约束 (UNIQUE(item_id)), 从数据库最底层保证同一商品绝不可能生成两条订单记录。

3. 如果系统崩溃, 如何恢复订单数据?

依赖 PostgreSQL 数据库的崩溃恢复机制。数据库在修改实际数据页前, 会先将操作记录写入 WAL 日志。即使系统突然断电崩溃, 重启后数据库也能通过重放 WAL 日志, 将已经 Commit 但未写入磁盘的订单数据恢复。定期备份与 PITR, 结合定期全量备份文件和归档的 WAL 日志, 如果发生灾难性故障, 可以将数据库恢复到崩溃发生前的任意一个准确时间点, 确保数据一致性和完整性不变。